



**MANIPAL INSTITUTE OF TECHNOLOGY**  
**MANIPAL**  
*(A constituent unit of MAHE, Manipal)*

## **SCHOOL OF COMPUTER ENGINEERING**

### **Embedded Systems Lab Manual [ICT 3144]**

**Semester: V  
B. Tech (IT)**

---

## **CONTENTS**

| <b>LAB NO</b> | <b>TITLE</b>                                  | <b>PAGENO.</b> | <b>MARKS</b> | <b>SIGN</b> |
|---------------|-----------------------------------------------|----------------|--------------|-------------|
|               | COURSE OBJECTIVES AND OUTCOMES                | II             |              |             |
|               | INSTRUCTION TO THE STUDENTS                   | III            |              |             |
| 1             | START- UP KEIL uVISION4                       | 1              |              |             |
| 2             | DATA TRANSFER PROGRAMS                        | 10             |              |             |
| 3             | ARITHMETIC PROGRAMS                           | 12             |              |             |
| 4             | CODE CONVERSION PROGRAMS                      | 17             |              |             |
| 5             | PROGRAMS ON SORTING, SEARCHING AND STACK      | 19             |              |             |
| 6             | INTERFACING TO ARM MICROCONTROLLER            | 21             |              |             |
| 7             | PROGRAMMING WITH LEDS                         | 24             |              |             |
| 8             | PROGRAMS ON MULTIPLEXED SEVEN SEGMENT DISPLAY | 26             |              |             |
| 9             | LCD AND KEYBOARD INTERFACING                  | 33             |              |             |
| 10            | ADC INTERFACING                               | 41             |              |             |
| 11            | PWM INTERFACING                               | 44             |              |             |
| 12            | MINI PROJECT                                  |                |              |             |
|               | APPENDIX A                                    | 62             |              |             |
|               | APPENDIX B                                    | 65             |              |             |
|               | APPENDIX C                                    | 69             |              |             |

## **Course Objectives**

- To develop skills in real world interfacing circuits.
- To efficiently design software for embedded systems.
- To design software for IoT applications.

## **Course Outcomes**

At the end of this course, students will be able to

- Design real world interfacing circuits to a microcontroller
- Develop software for embedded systems.
- Propose architectural solutions for IoT applications.
- Develop software for IoT applications.

## **INSTRUCTIONS TO THE STUDENTS**

### **Pre- Lab Session Instructions**

1. Students should carry the Lab Manual Book and the required stationery to every lab session.
2. Be in time and follow the institution dress code.
3. Must sign in the log register provided.
4. Make sure to occupy the allotted seat and answer the attendance.
5. Adhere to the rules and maintain the decorum.

### **In- Lab Session Instructions**

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- On receiving approval from the instructor, write the program and results in the lab record.
- Prescribed textbooks and class notes can be kept ready for reference, if required.

### **General Instructions for the exercises in Lab**

- Implement the given exercise individually as well as in allotted teams.
- The programs should meet the following criteria:
  - Programs should be interactive with appropriate prompt messages, error messages if any, and descriptive messages for outputs.

- Comments should be used to give the statement of the problem.
  - Statements within the program should be properly indented.
- Plagiarism (copying from others) is strictly prohibited and would invite severe penalty during evaluation.
- The exercises for each lab are divided under three sets:
  - Solved exercise,
  - Lab exercises - to be completed during lab hours,
  - Additional Exercises - to be completed outside the lab or in the lab to enhance the skill.
- In case a student misses a lab class, he/ she must ensure that the experiment is completed during the repetition class with the permission of the faculty concerned, but credit will be given only to one day's experiment(s).

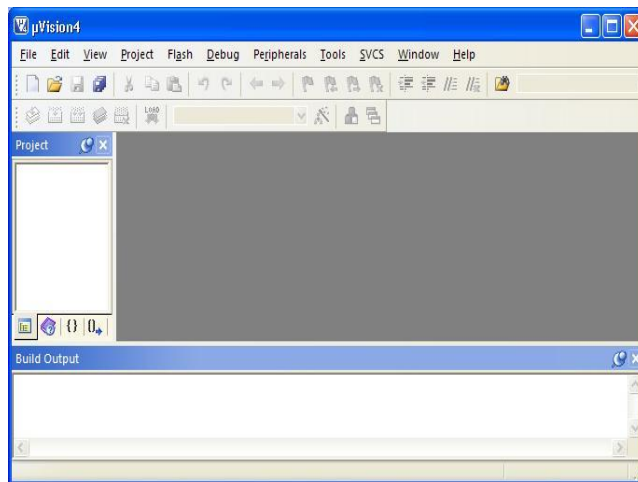
## Start up Keil uVision4

**Aim:** Understand the usage of Keil u Vision 4 software for assembly language.

Before you start up, you are recommended that you create a folder to hold all your project files. For example: you can create a folder "FirstARM-Project" ready before hand.

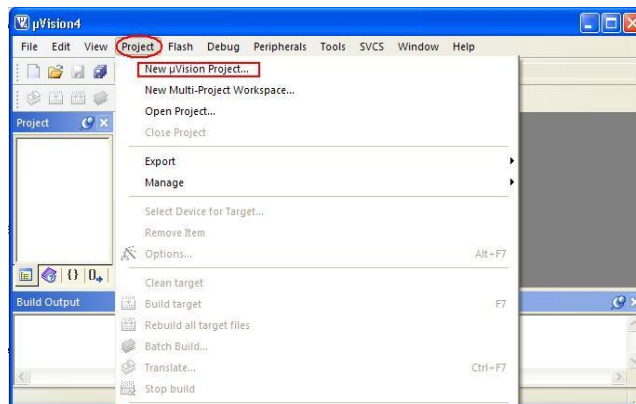
### Step1:

You can start up uVision4 by clicking on the icon  from the desktop or from the "Start" menu or "All Programs" on a lab PC. The following screen is what you will see:

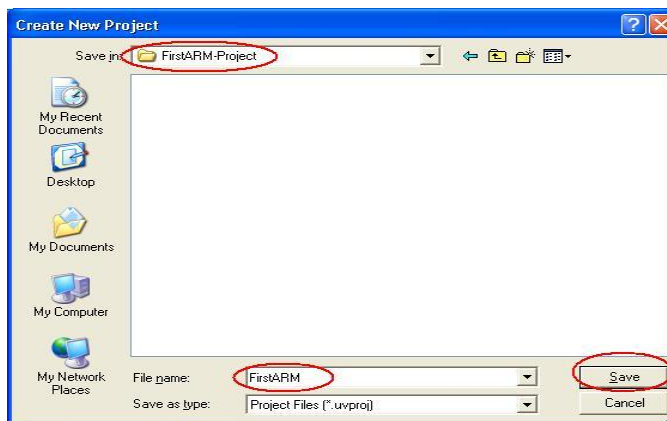


### Step2: Create a project

To create a project, click on the "Project" menu from the uVision4 screen and select "New uVision Project...".



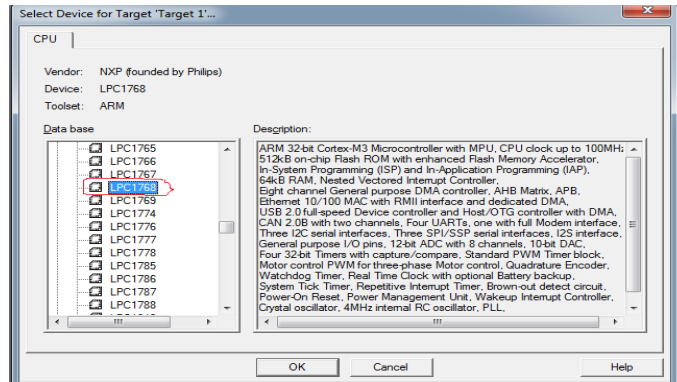
Then, select a folder, give project a name and save.



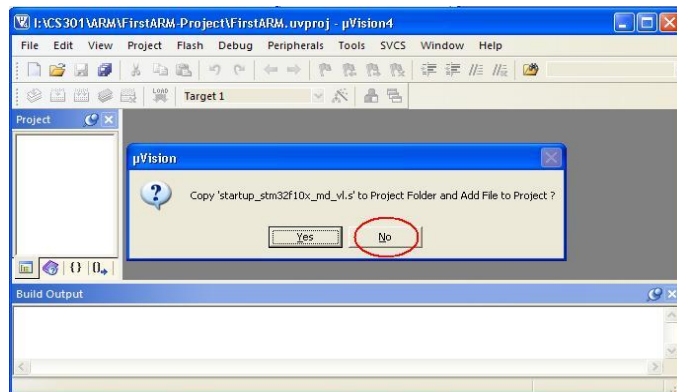
From the "Select Device for Target" window, select "NXP" as the vendor. In that select LPC1768 ARM controller , then click on OK button

### Caution

When you choose the chip some general information of the chip is shown in the **Description** box.

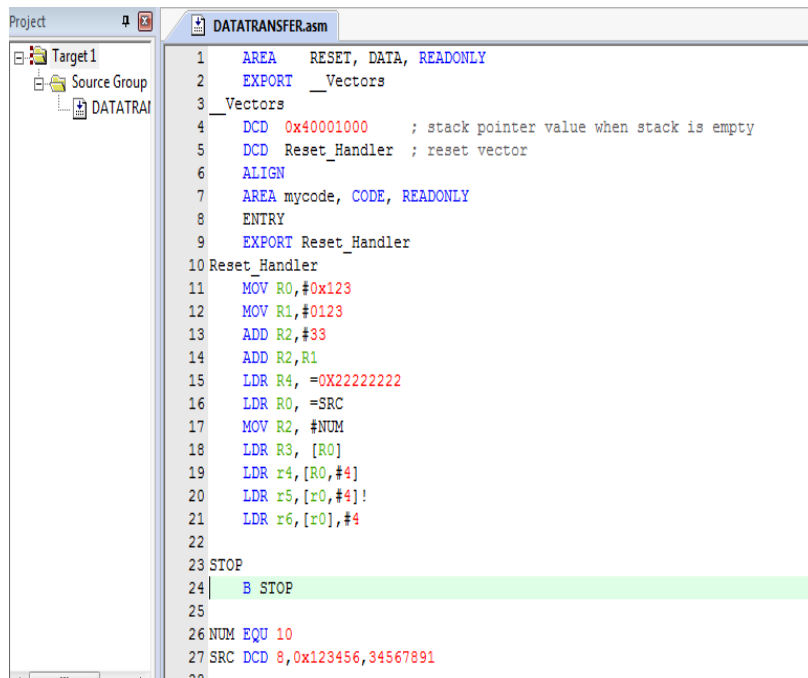


Make sure you click on "NO" for the following pop up window.



### Step3: Create Source File

From the "File" menu, select "New", you will see the "Text1\*" text edit window. That is the place you will write your ARM Assembly language program. You can write the program into this window. (Note: give a tab space at the beginning)



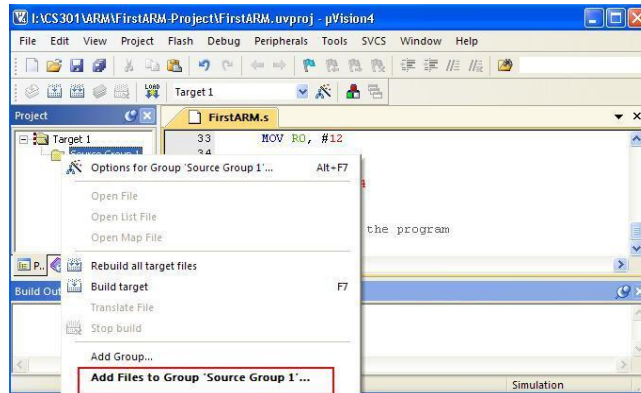
Save the program by clicking on the "Save" or "Save As" from the "File" menu and give it a name.



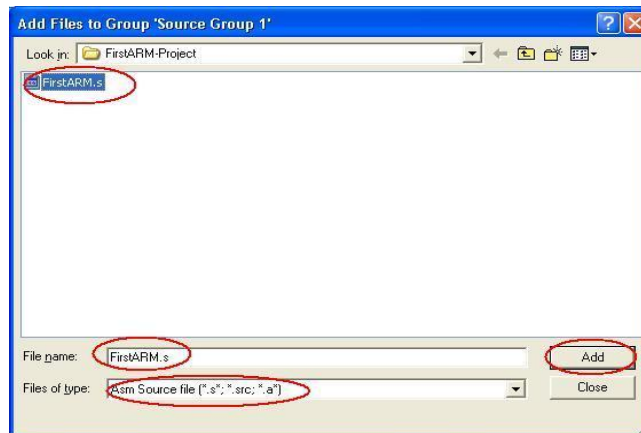


## Add Source File to the Project

Right click on the "Source Group 1", select "Add Files to Group 'Source Group 1'".

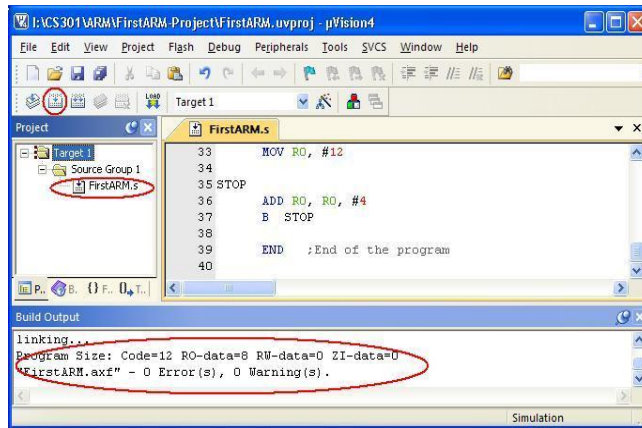


Select "Files of type" as "asm Source file (\*.s\*;\*.src\*;\*.a\*)", then select the file "FirstARM.s" for example. Click on "Add", and then click on "Close".



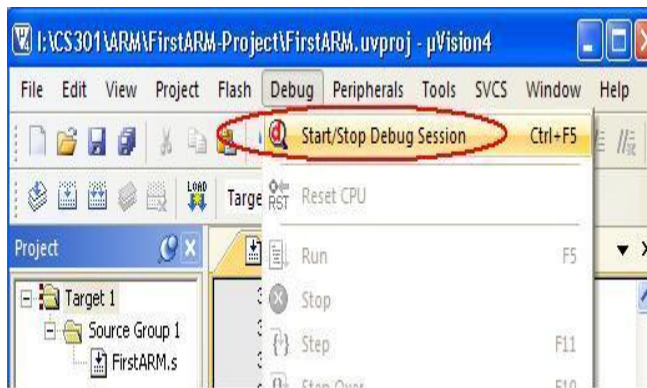
## Step4: Build your project

Click on the "+" beside the "Source Group 1", you will see the program "FirstARM.s". Click on the "Build" button or from the "Project" menu, you will see the following screen.



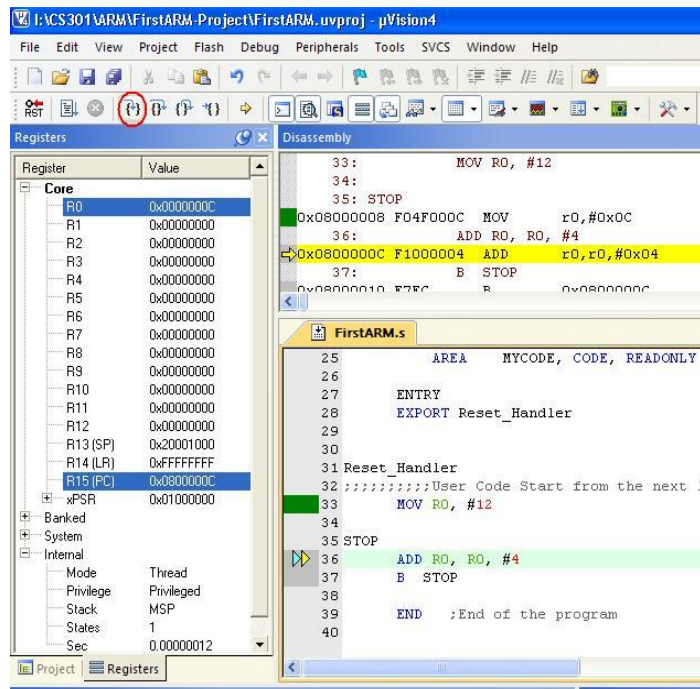
### Run the program in your project

Run the program through "Debug" menu.

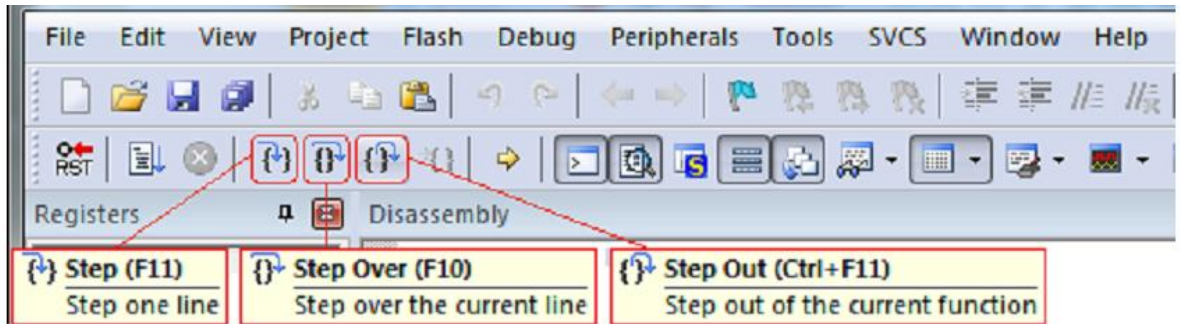


Click on "OK" for the pop up window showing "EVALUATION MODE, Running with Code Size Limit: 32K".

Open uVision4 to full screen to have a better and complete view. The left hand side window shows the registers and the right side window shows the program code. There are some other windows open. Adjust the size of them to have better view. Run the program step by step; observe the change of the values in the registers.



To trace the program use the **Step Over** button or click on **Step Over** from the Debug menu. It executes the instructions of the program one after another. To trace the program one can use the **Step** button, as well. The difference between the **Step Over** and **Step** is in executing functions. While **Step** goes into the function and executes its instructions one by one, **Step Over** executes the function completely and goes to the instruction next to the function. To see the difference between them, trace the program once with **Step Over** and then with **Step**. When PC executing the function and want the function to be executed completely one can use **Step Out**. In the case, the instructions of the function will be executed, it returns from the function, and goes to the instruction which is next to the function call.



Click on the "Start/Stop Debug Session" again to stop execution of the program.

## An example ARM assembly language module

An ARM assembly language module has several constituent parts.

These are:

- Extensible Linking Format (ELF) sections (defined by the `AREA` directive).
- Application entry (defined by the `ENTRY` directive).
- Application execution.
- Program end (defined by the `END` directive).

Consider the following example

```

▪ AREA      ARMex, CODE, READONLY
▪
▪                               ; Name this block of code ARMex
▪      ENTRY                               ; Mark first instruction to execute
▪      start
▪      MOV      r0, #10           ; Set up parameters
▪      MOV      r1, #3
▪      ADD      r0, r0, r1       ; r0 = r0 + r1
▪      END                               ; Mark end of file

```

**Application entry**

The ENTRY directive declares an entry point to the program. It marks the first instruction to be executed. In applications using the C library, an entry point is also contained within the C library initialization code. Initialization code and exception handlers also contain entry points.

**Application execution**

The application code begins executing at the label start, where it loads the decimal values 10 and 3 into registers R0 and R1. These registers are added together and the result placed in R0.

**Program end**

The END directive instructs the assembler to stop processing the source file. Every assembly language source module must finish with an END directive on a last line. Any lines following the END directive are ignored by the assembler.

## LAB 2:DATA TRANSFER PROGRAMS

**Aim:** Familiarization of ARM data transfer instructions.

**Introduction to ARM addressing modes:** Data can be transferred into and out of ARM controller using different addressing modes. There are different ways to specify the address of the operands for any given operations such as load, add or branch. The different ways of determining the address of the operands are called addressing modes. In this lab, we are going to explore different data transfer instructions of ARM processor and learn how all instructions can fit into a single word (32 bits).

Appendix A gives the different addressing modes used in ARM

**Question:** Write a ARM assembly language program to copy 16 bit variable from code memory to data memory.

Input : SRC = 0X00000008 at location pointed by R0

Output : DST = 0X00000008 At location pointed by R1

```
AREA RESET, DATA, READONLY
EXPORT __Vectors

__Vectors
    DCD 0x10001000 ; stack pointer value when stack is empty
    DCD Reset_Handler ; reset vector

ALIGN

AREA mycode, CODE, READONLY
ENTRY
EXPORT Reset_Handler

Reset_Handler
    LDR R0, =SRC;      Load address of SRC into R0
    LDR R1, =DST;      Load the address of DST onto R1
```

```

    LDR R3, [R0];      Load data pointed by R0 into R3
    STR R3,[R1]  ;      Store data from R3 into the address pointed by R1
STOP
    B STOP

SRC DCD 8,0x123456;    SRC location in code segment
    AREA DATASEG, DATA, READWRITE

DST DCD 0              ;DST location in Data segment

END

```

### Observations to be made

1. **Data storage into the memory:** Click on Memory window you get label Memory1 option type address pointed by R0 in address space and observe how the data are stored into the memory.
2. **Data movement from one memory to another memory:** Click on Memory window you get label Memory2 option type address pointed by R1 in address space and observe data movement to another location before execution and after execution.

### Exercise questions

1. Write an ARM assembly language program to transfer block of ten 32 bit numbers from one memory to another
  - a. When the source and destination blocks are non-overlapping
  - b. When the source and destination blocks are overlapping
 Hint: Use Register indirect addressing mode or indexed addressing mode
2. Reverse an array of ten 32 bit numbers in the memory.

## Lab 3: Arithmetic Programs

**Aim:** Familiarization

ion of Arithmetic operations - addition and subtraction, multiplication

Please refer to appendix A for arithmetic instructions.

**Question:** Write a program to add two 32 bit numbers.

```

        AREA  RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD  0x10001000    ; stack pointer value when stack is empty
        DCD  Reset_Handler ; reset vector

        ALIGN
        AREA mycode, CODE, READONLY
        ENTRY
        EXPORT Reset_Handler

Reset_Handler
        LDR R0, =VALUE1    ;pointer to the first value1
        LDR R1,[R0]         ;load the first value into R1
        LDR R0,=VALU2      ;pointer to the second value
        LDR R3, [R0]        ;load second number into r3
        ADDS R6, R1,R3      ;add two numbers and store the result in r6
        LDR R2, =RESULT
        STR R6,[R2]

        STOP
        B STOP

VALUE1 DCD 0X12345678      ; First 32 bit number
VALUE2 DCD 0XABCDEF12      ; Second 32 bit number
        AREA data, DATA, READWRITE
        RESULT DCD 0

```



**Question:** Write an assembly program to multiply two 32 bit numbers

```

AREA  RESET, DATA, READONLY
EXPORT  __Vectors

__Vectors
    DCD  0x10001000    ; stack pointer value when stack is empty
    DCD  Reset_Handler ; reset vector

ALIGN

AREA mycode, CODE, READONLY
ENTRY
EXPORT Reset_Handler

Reset_Handler
    LDR R0, =VALUE1    ;pointer to the first value1
    LDRH R1,[R0]        ;load the first value into r1
    LDR R0,=VALU2      ;pointer to the second value
    LDRH R3, [R0]       ;load  second number into r3
    MUL R6, R1,R3       ;Multiply the values from R1 and R3 and store
                        ;least significant 32 bit number into R6.

    LDR R2, =RESULT
    STR R6,[R2]         ; store result in r6
    STOP
    B STOP
VALUE1 DCD 0X1234; First 32 bit number
VALUE2 DCD 0X5678; Second 32 bit number
    AREA data, DATA, READWRITE
    RESULT DCD 0

```

**Note: If the result is more than 32 bits, use UMULL instruction.**

Question: Write a program to divide a 32 bit number by 16 bit number

- . We follow repetitive subtraction method to divide two numbers.

```

AREA  RESET, DATA, READONLY
EXPORT  __Vectors

```

```
__Vectors
```

```

    DCD  0x10001000    ; stack pointer value when stack is empty
    DCD  Reset_Handler ; reset vector

```

```
ALIGN
```

```
AREA mycode, CODE, READONLY
```

```
ENTRY
```

```
EXPORT Reset_Handler
```

```
Reset_Handler
```

```

    MOV R2,#00
    LDR R0, =VALUE1    ;pointer to the first value1
    LDR R1,[R0]         ;load the first value into r1
    LDR R0,=VALUE2     ;pointer to the second value
    LDR R3, [R0]        ;load second number into r3
up    SUB R1, R3        ;Subtract two numbers
    ADD R2,#01         ;increment a counter
    CMP R1,R3          ;compare two numbers
    BCS up             ;check R1 is greater than R3 or not, if yes loop
    LDR R6, =RESULT    ;Quotient
    STR R2,[R6,#4]
    STR R1,[R6]        ;Store remainder.

```

```
STOP
```

```
B STOP
```

```
VALUE1 DCD 0x200000000 ;First 32 bit number
```

```
VALUE2 DCD 0x00050000 ;Second 16 bit number
```

```
AREA data, DATA, READWRITE
```

```
RESULT DCD 0,0
```

### Exercise questions

1. Write a program to add ten 32 bit numbers stored in code segment and store the result in data segment

2. Write a program to add two 128 bit numbers stored in code segment and store the result in data segment.

Hint: Use indexed addressing mode.

3. Write a program to subtract two 128 bit numbers

4. Write an assembly language program to find LCM of two numbers

Hint: i=1

```
do{
    remainder= i*a mod b;
    If (remainder==0)
        Exit;
    Else
        i++;
    } while(remainder!=0);
Return (i*a);
```

#### Additional Exercise questions

1. Write a program to subtract two 32 bit numbers
2. Find the sum of 'n' natural numbers using MLA instruction.
3. Write an assembly language program to find GCD of two numbers

Hint:

```
While(a!=b)
{
    If(a>b)
        a=a-b;
    else
        b=b-a;
} Return (a);
```

4. Write a program to multiply two 32 bit numbers using repetitive addition

Hint: If two numbers are in R0 and R1 Registers then use following algorithm

```
Sum=0;
do { sum=sum+R0; R1--; ;Use ADS instruction for addition and use
ADD                                     ;instruction to increment a register by 1
if carry then
```

```
        R2++;                ;Increment carry value by one.
    } while(R1!=0);          ;Use Compare instruction to check
greater                      ;than or not. And Brach instructions for
loop
Result= R2 and R0
```

5. Write a program for BCD multiplication

## Lab 4

### Code Conversion Programs

**Aim:** Familiarization of logical instructions and code conversion programs.

**Question:** Write an assembly program to convert a 2 digit hexadecimal number into unpacked ASCII.

```
AREA  RESET, DATA, READONLY
EXPORT __Vectors
```

```
__Vectors
```

```
    DCD 0x10001000    ; stack pointer value when stack is empty
    DCD Reset_Handler ; reset vector
    ALIGN
    AREA mycode, CODE, READONLY
    ENTRY
    EXPORT Reset_Handler
```

```
Reset_Handler
```

```
    LDR R0,=NUM
    LDR R3,=RESULT
    LDRB R1,[R0]        ; load hex number into register R1
    AND R2,R1,#0x0F     ; mask upper 4 bits
    CMP R2,#09          ; compare the digit with 09
    BLO DOWN            ; if it is lower than 9 then jump to down
                        ; lable
    ADD R2,#07          ;else add 07 to that number
DOWN
    ADD R2,#0x30         ; Add 30H to the number, Ascii value of first
    STRB R2,[R3]         ; digit
    AND R3,R1,#0xF0      ; Mask the second digit
    MOV R3,R3,LSR#04     ; Shift right by 4 bits
```

```
        CMP R3,#09          ; check for >9 or not
        BLO DOWN1
        ADD R3,#07
DOWN1
        ADD R3,#0x30        ; Ascii value of second digit
        STRB R3,[R2,#01]
        END
NUM DCD 0x000003A
        AREA data, DATA, READWRITE
RESULT DCD 0
```

### Exercise programs

1. Write an ARM assembly language program to covert 2-digit hexadecimal number in ASCII
2. Write an assembly language program to convert a 2-digit BCD number in to its equivalent hexadecimal number.
3. Write an assembly language program to convert a 2-digit hex number in to its equivalent BCD number

### Additional Exercise

1. Write an ARM assembly language program to covert 2-digit hexadecimal number in ASCII unpacked form into its equivalent packed hexadecimal number
2. Write a program to convert a 32 bit number in the unpacked form into packed form.

## Lab 5 : Programs on Sorting, Searching and Stack

**Aim:** To understand the logic of looping and sorting

**Question:** Write an ARM program to sort a list using bubble sort.

```

        AREA    RESET, DATA, READONLY
        EXPORT  __Vectors

__Vectors
        DCD  0x10001000    ; stack pointer value when stack is empty
        DCD  Reset_Handler ; reset vector

        ALIGN
        AREA ascend, code, readonly
        ENTRY
Reset_Handler
        mov r4, #0
        mov r1, #10
        ldr r0, =list
        ldr r2, =result
up       ldr r3, [r0, r4]
        str r3, [r2, r4]
        add r4, #04
        sub r1, #01
        cmp r1, #00
        bhi up
        ldr r0, =result

                                ; inner loop counter
        mov r3, #10
        sub r3, r3, #1
        mov r9, r3
                                ; R9 contain no of passes
                                ; outer loop counter
outer_loop
        mov r5, r0

```

```

        mov r4, r3                ; R4 contains no of compare in a pass
inner_loop
        ldr r6, [r5], #4
        ldr r7, [r5]
        cmp r7, r6

                                ; swap without swp instruction
        strls r6, [r5]
        strls r7, [r5, #-4]

        subs r4, r4, #1
        bne inner_loop
        sub r3, #1
        subs r9, r9, #1
        bne outer_loop

list dcd 0x10,0x05,0x33,0x24,0x56,0x77,0x21,0x04,0x87,0x01
        AREA data1, data, readwrite
        result DCW 0,0,0,0,0,0,0,0,0,0
        end

```

**Exercise questions:**

1. Write an assembly program to sort an array using selection sort
2. Write an assembly program to find the factorial of a unsigned number using recursion
3. Write an assembly program to search an element in an array of ten 32 bit numbers using linear search.

**Additional Exercise**

1. Assume that ten 32 bit numbers are stored in registers R0-R10. Sort these numbers in the fully ascending stack using selection sort and store the sorted array back into the registers. Use STM and LDMDb instructions wherever necessary.
2. Repeat the above question (4) for fully descending stack using STMDb and LDM instruction wherever necessary.



## Lab 6 : Interfacing LED to ARM Microcontroller

**Aim:** Interface LEDs and switch to the ARM cortex LPC1768 microcontroller using ALS interfacing board.

### Steps to be followed:

#### Project Creation in Keil uvision4 IDE:

- Create a project folder before creating NEW project.
- Use separate folder for each project
- Open Keil uVision4 IDE software by double clicking on “Keil Uvision4” icon.
- Select “Project” then to “New Project” and save it with a name in the respective Project folder, which is already you created.
- Select the device as “NXP (founded by Philips)” Select “LPC1768” then Press “OK” and then press “YES” button to add “system\_LPC17xx.s” file.
- Go to “File” select “New” to open an editor window. Create a source file and use the header file “LPC17xx.h” in the source file and save the file. Color syntax highlighting will be enabled once the file is saved with a Recognized extension such as “.C”.
- Right click on “Source Group 1” and select the option “Add Files to Group 'Source Group 1' “add the. C source file(s) to the group.
- Again right click on Source Group 1 and select the option “Add Files to Group 'Source Group 1' “add the file - C:Keil\ARM\startup\NXP\LPC17xx\system\_LPC17xx.c
- Any changes made to this file at current project will directly change the source system\_LPC17xx.C file. As a result other project settings may get altered. So it is recommended to copy the file C:Keil\ARM\startup\NXP\LPC17xx\system\_LPC17xx.c to the project folder and add to the source group.
- Important: This file should be added during each project creation.
- Select “Project” then select “Translate” to compile the File (s).
- Select “Project”, select “Build Target” for building all source files such as “.C”, “.ASM”, “.h”, files, etc... This will create the hex file if there are no warnings & no errors.

**Some Settings to be done in KEILUV4 for Executing C programs :**

- In Project Window Right click “TARGET1” and select “options for target ‘TARGET1’” Then go to option “Output” tab and Select “Create Hex file”.

**Components required**

- ALS-SDA-ARMCTXM3-01 : 1 No.
- Power supply (+5V) : 1 No.
- Cross cable for programming and serial communication : 1 No
- One working USB port in the host computer system and PC for downloading the software.
- 10 core FRC cables of 8 inch length 2 No
- USB to B type cable 1 No

**Some Settings for downloading the program in FLASH MAGIC:**

Step1.Connect 9 pin DSUB cross cable from PC to CN9 at the board.

Step2.On the 2 way dip switch SW21. Short jumper JP3

Step3.Open flash magic 6.01

Step4.Make following setting in Flash magic(Only once)

**a. Communications:**

Device: LPC1768

Com Port: depends on the COM port to which USB cable is inserted

Baud Rate: 9600

Interface: None(ISP)

Oscillator: 12MHz

**b. ERASE:**

Select “Erase Blocks Used by Hex File”.

**c. Hex file:**

Browse and select the Hex file which you want to download.

**d. Options:**

Select “Verify After Programming”.

**Go to Options -> Advanced Options->communications.**

Do not select High Speed Communications, keep baud rate 115200.

**Options -> Advanced Options->Hardware config**

Select Use DTR & RTS to control RST & ISP Pin.

Select Keep RTS asserted while COM Port open.

T1 = 50ms. T2 = 100ms.

Step5.Start:

Click “Start” to download the hex file to the controller.

Step6. Connect one end of 10 pin FRC cable to CNA1, Short other end to CNA

Step7. Press reset controller switch SW1 and Check output on the LEDs  
connected to CNA1.

**Exercise Questions:**

1. Write an embedded C program to blink an LED.
2. Write an embedded C program to turn ON the LED when the switch connected to P2.12 is pressed and turn OFF the LED when the switch is released.

## LAB 7 Program on LED Interfacing

**Aim:** To learn advanced programming on interfacing the LEDs and switch to the ARM cortex LPC1768 microcontroller using ALS interfacing board.

**Sample program to turn on/off LED serially.**

**Note:** Before writing the program please check GPIO port pins available in the kit (Refer Appendix C.)

```
#include <LPC17xx.h>

unsigned int i,j;
unsigned long LED = 0x00000010;

int main(void)
{
    SystemInit()           ;Add these two function for its
                           ;internal operation

    SystemCoreClockUpdate();

    LPC_PINCON->PINSEL0 &= 0xFF0000FF
                           ;Configure Port0 PINS P0.4-P0.11
                           ;as GPIO function

    LPC_GPIO0->FIODIR |= 0x00000FF0;
                           ;Configure P0.4-P0.11 as output
                           ;port

    while(1)
    {
        LED = 0x00000010; Initial value on LED
        for(i=1;i<9;i++)    //On the LED's serially
        {
```

```

LPC_GPIO0->FIOSET = LED;

        ; Turn ON LED at LSB(LED
        ;connected ;to p0.4)

for(j=0;j<10000;j++);a random delay
    LED <<= 1; Shift the LED to the left by one
        ;unit.

    }
        ; loop for 9 times

LED = 0x00000010;

for(i=1;i<9;i++) //Off the LED's serially
{
    LPC_GPIO0->FIOCLR = LED;
        ;Turn OFF LED at LSB(LED
        connected ;to p0.4)
    for(j=0;j<10000;j++);
    LED <<= 1;
}

    }

}

```

**Exercise Questions:**

1. Write a C program to display 8-bit binary up counter on the LEDs.
2. Write a C program to read a key and display an 8-bit up/down counter on the LEDs.  
**Hint:** Use key SW2(if SW2=1, up counter else down counter), which is available at CNB1 pin 7. Connect CNB1 to any controller connector like CNB, CNC etc. Configure corresponding port pin as GPIO using corresponding PINSEL register and input pin using corresponding FIODIR register.

**Additional Exercise:**

3. Write a program to simulate an 8- bit ring counter with key press (SW2).

## Lab 8

### Programs on Multiplexed Seven Segment Display

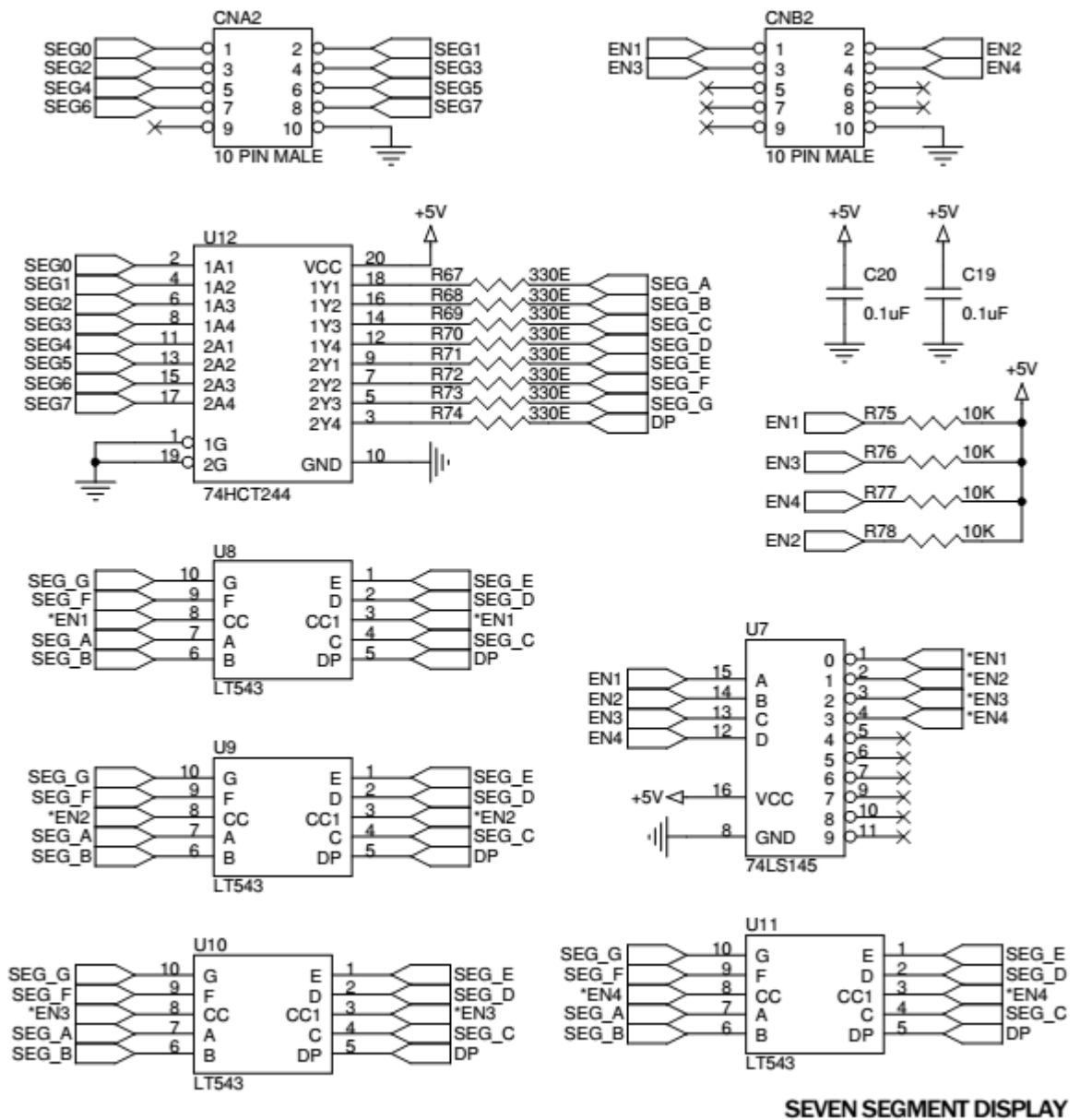
**Aim:** To interface and understand the working of multiplexed seven segments display.

#### **Introduction:**

There are four multiplexed 7-segment display units (U8, U9, U10 and U11) on the board. Each display has 8-inputs SEG\_A (Pin-7), SEG\_B (Pin-6), SEG\_C (Pin-4), SEG\_D (Pin-2), SEG\_E (Pin-1), SEG\_F (Pin-9), SEG\_G (Pin-10) and SEG\_H (Pin-5) and the remaining pins pin-3 & pin-8 are Common Cathode CC. These segments are common cathode type hence active high devices.

At power on all the segments are pulled up. A four bits input through CNB2 is used for multiplexing operation. A 1-of-10 Decoder/Driver U7 is used to accept BCD inputs and provide appropriate outputs for enabling the required display.

8 bits data is provided in this block using CNA2. All the data lines are taken buffered at U12 before giving to the displays.



At controller end any 2 connector are required for interfacing this block.

Lookup Table for displaying 0,1,2,3 to 9

| value=   | h | g | f | e | d | c | b | a | On 7-SEG U8,U9,U10 & U11. |
|----------|---|---|---|---|---|---|---|---|---------------------------|
| * 0x3F = | 0 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | -> Displaying '0'         |
| * 0x06 = | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | -> Displaying '1'         |
| * 0x5B = | 0 | 1 | 0 | 1 | 1 | 0 | 1 | 1 | -> Displaying '2'         |
| * 0x4F = | 0 | 1 | 0 | 0 | 1 | 1 | 1 | 1 | -> Displaying '3'         |
| * 0x66 = | 0 | 1 | 1 | 0 | 0 | 1 | 1 | 0 | -> Displaying '4'         |
| * 0x6D = | 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | -> Displaying '5'         |
| * 0x7D = | 0 | 1 | 1 | 1 | 1 | 1 | 0 | 1 | -> Displaying '6'         |
| * 0x07 = | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | -> Displaying '7'         |
| * 0x7F = | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | -> Displaying '8'         |
| * 0x6F = | 0 | 1 | 1 | 0 | 1 | 1 | 1 | 1 | -> Displaying '9'         |

**Sample program :** To simulate 4-digit BCD up counter on the multiplexed seven segment display.

```
#include <LPC17xx.h>
#include <stdio.h>

#define FIRST_SEG 0xF87FFFFFFF
#define SECOND_SEG 0xF8FFFFFFF
#define THIRD_SEG 0xF97FFFFFFF
#define FOURTH_SEG 0xF9FFFFFFF
#define DISABLE_ALL 0xFA7FFFFFFF

unsigned int dig1=0x00,dig2=0x00,dig3=0x00,dig4=0x00;
unsigned int twenty_count = 0x00,dig_count=0x00,temp1=0x00;
unsigned char
array_dec[10]={0x3F,0x06,0x5B,0x4F,0x66,0x6D,0x7D,0x07,0x7F,0x6F
};
unsigned char tmr0_flg = 0x00,one_sec_flg = 0x00;
unsigned long int temp2 = 0x00000000,i=0;
unsigned int temp3=0x00;
void delay(void);
void display(void);

int main(void)
{
    SystemInit();
    SystemCoreClockUpdate();
```



```
LPC_PINCON->PINSEL0 &= 0xFF0000FF;    //P0.4 to P0.11
                                       //GPIO data lines
LPC_PINCON->PINSEL3 &= 0xFFC03FFF;    //P1.23 to P1.26
                                       //GPIO enable lines
```

```
LPC_GPIO0->FIODIR |= 0x00000FF0; //P0.4 to P0.11 output
LPC_GPIO1->FIODIR |= 0x07800000; //P1.23 to P1.26 output
```

```
while(1)
{
    Delay();
    dig_count +=1;
    if(dig_count == 0x05)
        { dig_count = 0x00;
          }

    if(one_sec_flg == 0xFF)
    {
        one_sec_flg = 0x00;
        dig1 +=1;

        if(dig1 == 0x0A)
        {
            dig1 = 0;
            dig2 +=1;

            if(dig2 == 0x0A)
            {
                dig2 = 0;
                dig3+=1;

                if(dig3 == 0x0A)
                {
                    dig3 = 0;
                    dig4 += 1;

                    if(dig4 == 0x0A)
                    {
                        dig4 = 0;
                    } //end of dig4

                } //end of dig3

            } //end of dig2
        }
    }
}
```

```
        } //end of dig1

    } //end of one_sec if

    Display();

} //end of while(1)

} //end of main

void Display(void) //To Display on 7-segments
{

    if(dig_count == 0x01) // For Segment U8
    {
        temp1 = dig1;
        LPC_GPIO1->FIOPIN = FIRST_SEG;
    }

    else if(dig_count == 0x02) // For Segment U9
    {
        temp1 = dig2;
        LPC_GPIO1->FIOPIN = SECOND_SEG;
    }

    else if(dig_count == 0x03) // For Segment U10
    {
        temp1 = dig3;
        LPC_GPIO1->FIOPIN = THIRD_SEG;
    }
    else if(dig_count == 0x04) // For Segment U11
    {
        temp1 = dig4;
        LPC_GPIO1->FIOPIN = FOURTH_SEG;
    }

    temp1 &= 0x0F;
    temp2 = array_dec[temp1]; // Decoding to 7-segment
    temp2 = temp2 << 4;
    LPC_GPIO0->FIOPIN = temp2; // Taking Data Lines for 7-Seg
```

```

        for(i=0;i<500;i++);
        LPC_GPIO0->FIOCLR = 0x00000FF0;
//      LPC_GPIO1->FIOPIN = DISABLE_ALL;//disable all the segments
    }
Void delay(void)
{   unsigned int i;
    For(i=0;i<1000;i++);
    if(twenty_count ==1000)        //multiplied by 500x2msec for
                                    //1 Sec
    {
        one_sec_flg = 0xFF;
        twenty_count = 0x00;
    }
    else twenty_count += 1;
}

```

### Components required

- |                                                                                    |       |
|------------------------------------------------------------------------------------|-------|
| • ALS-SDA-ARMCTXM3-01 :                                                            | 1 No. |
| • Power supply (+5V) :                                                             | 1 No. |
| • Cross cable for programming and serial communication :                           | 1 No  |
| • One working USB in the host computer system and PC for downloading the software. |       |
| • 10 core FRC cables of 8 inch length                                              | 2 No  |
| • USB to B type cable                                                              | 1 No  |

**Hardware setup:** Connect a 10 core FRC cable from CNA to CNA2 and CNB to CNB2.

**Working procedure:** After software download and hardware setup, press the reset, Observe the count from 0000 to 9999 on the display.

### Exercise Questions:

1. Write a C program for 4 digit BCD up/down counter on seven segment using a switch and timer with a delay of 1-second between each count.
2. Write a program to display 4-digit binary ring counter on the multiplexed seven segment display.

**Additional Exercise:**

1. Write a C program to simulate a 4 digit BCD down counter. Use timer for a delay
2. Write a program for 4 digit Hexadecimal up/down counter on seven segment using a switch and timer with a delay of 1-second between each count

## Lab 9

### Liquid Crystal Display (LCD) and Keyboard Interfacing

**Aim:** To interface and understand the working of LCD and matrix keyboard.

#### Introduction:

**LCD:** A 16×2 alphanumeric LCD can be used to display the message from controller.

16 pin small LCD has to be mounted to the connector CN11. 10 pin connector CNAD is used to interface this LCD from controller. Only higher 4 data lines are used among the 8 LCD data lines. Use POT3 for contrast adjustment and Short the jumper JP16 to use this LCD. LCD connector CN11 is described in this table. CN11 is single row 16 pin female berg.

| Pin no CN11 | Description        |
|-------------|--------------------|
| 1           | Ground             |
| 2           | +5V                |
| 3           | LCD contrast       |
| 4           | RS                 |
| 5,7,8,9,10  | NC                 |
| 6           | En                 |
| 11 to 14    | Data 4 to 7        |
| 15          | Back light anode   |
| 16          | Back light cathode |

Connection from CNAD to LCD connector CN11 is shown below

| Pin no at CNAD | Description              | Pin no at CN11 |
|----------------|--------------------------|----------------|
| 1              | L0 – Data line 4 of LCD  | 11             |
| 2              | L1 – Data line 5 of LCD  | 12             |
| 3              | L2 – Data line 6 of LCD  | 13             |
| 4              | L3 – Data line 7 of LCD  | 14             |
| 5              | L5 – Command line of LCD | 4              |
| 6              | L5 – Enable line of LCD  | 6              |

| Instruction                | Code                                                                                                                                                                                                                                                                                                                                                                                     |     |            |     |     |     |     |     |     |     | Description                                                                                                                                                                                          | Execution Time<br>(max) (when $f_{cp}$ or<br>$f_{osc}$ is 270 kHz)                                                                                    |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|------------|-----|-----|-----|-----|-----|-----|-----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | RS                                                                                                                                                                                                                                                                                                                                                                                       | R/W | DB7        | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |                                                                                                                                                                                                      |                                                                                                                                                       |
| Write data to CG or DDRAM  | 1                                                                                                                                                                                                                                                                                                                                                                                        | 0   | Write data |     |     |     |     |     |     |     | Writes data into DDRAM or CGRAM.                                                                                                                                                                     | 37 $\mu$ s<br>$t_{ADD} = 4 \mu$ s*                                                                                                                    |
| Read data from CG or DDRAM | 1                                                                                                                                                                                                                                                                                                                                                                                        | 1   | Read data  |     |     |     |     |     |     |     | Reads data from DDRAM or CGRAM.                                                                                                                                                                      | 37 $\mu$ s<br>$t_{ADD} = 4 \mu$ s*                                                                                                                    |
|                            | I/D = 1: Increment<br>I/D = 0: Decrement<br>S = 1: Accompanies display shift<br>S/C = 1: Display shift<br>S/C = 0: Cursor move<br>R/L = 1: Shift to the right<br>R/L = 0: Shift to the left<br>DL = 1: 8 bits, DL = 0: 4 bits<br>N = 1: 2 lines, N = 0: 1 line<br>F = 1: 5 $\times$ 10 dots, F = 0: 5 $\times$ 8 dots<br>BF = 1: Internally operating<br>BF = 0: Instructions acceptable |     |            |     |     |     |     |     |     |     | DDRAM: Display data RAM<br>CGRAM: Character generator RAM<br>ACG: CGRAM address<br>ADD: DDRAM address<br>(corresponds to cursor address)<br>AC: Address counter used for both DD and CGRAM addresses | Execution time changes when frequency changes<br>Example:<br>When $f_{cp}$ or $f_{osc}$ is 250 kHz,<br>$37 \mu$ s $\times \frac{270}{250} = 40 \mu$ s |

Note: — indicates no effect.

- \* After execution of the CGRAM/DDRAM data write or read instruction, the RAM address counter is incremented or decremented by 1. The RAM address counter is updated after the busy flag turns off. In Figure 10,  $t_{ADD}$  is the time elapsed after the busy flag turns off until the address counter is updated.

| Instruction              | Code |     |     |     |     |     |     |     |     |     | Description                                                                                                                             | Execution Time<br>(max) (when $f_{cp}$ or<br>$f_{osc}$ is 270 kHz) |
|--------------------------|------|-----|-----|-----|-----|-----|-----|-----|-----|-----|-----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------|
|                          | RS   | R/W | DB7 | DB6 | DB5 | DB4 | DB3 | DB2 | DB1 | DB0 |                                                                                                                                         |                                                                    |
| Clear display            | 0    | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | Clears entire display and sets DDRAM address 0 in address counter.                                                                      |                                                                    |
| Return home              | 0    | 0   | 0   | 0   | 0   | 0   | 0   | 0   | 1   | —   | Sets DDRAM address 0 in address counter. Also returns display from being shifted to original position. DDRAM contents remain unchanged. | 1.52 ms                                                            |
| Entry mode set           | 0    | 0   | 0   | 0   | 0   | 0   | 0   | 1   | I/D | S   | Sets cursor move direction and specifies display shift. These operations are performed during data write and read.                      | 37 $\mu$ s                                                         |
| Display on/off control   | 0    | 0   | 0   | 0   | 0   | 0   | 1   | D   | C   | B   | Sets entire display (D) on/off, cursor on/off (C), and blinking of cursor position character (B).                                       | 37 $\mu$ s                                                         |
| Cursor or display shift  | 0    | 0   | 0   | 0   | 0   | 1   | S/C | R/L | —   | —   | Moves cursor and shifts display without changing DDRAM contents.                                                                        | 37 $\mu$ s                                                         |
| Function set             | 0    | 0   | 0   | 0   | 1   | DL  | N   | F   | —   | —   | Sets interface data length (DL), number of display lines (N), and character font (F).                                                   | 37 $\mu$ s                                                         |
| Set CGRAM address        | 0    | 0   | 0   | 1   | ACG | ACG | ACG | ACG | ACG | ACG | Sets CGRAM address. CGRAM data is sent and received after this setting.                                                                 | 37 $\mu$ s                                                         |
| Set DDRAM address        | 0    | 0   | 1   | ADD | ADD | ADD | ADD | ADD | ADD | ADD | Sets DDRAM address. DDRAM data is sent and received after this setting.                                                                 | 37 $\mu$ s                                                         |
| Read busy flag & address | 0    | 1   | BF  | AC  | AC  | AC  | AC  | AC  | AC  | AC  | Reads busy flag (BF) indicating internal operation is being performed and reads address counter contents.                               | 0 $\mu$ s                                                          |

**Sample program: To display message on LCD**

```

#include <lpc17xx.h>
#define RS_CTRL 0x08000000 //P0.27, 1<<27
#define EN_CTRL 0x10000000 //P0.28, 1<<28
#define DT_CTRL 0x07800000 //P0.23 to P0.26 data lines, F<<23

unsigned long int temp1=0, temp2=0,i,j ;
unsigned char flag1 =0, flag2 =0;
unsigned char msg[] = {"MITn 01manipal"};

void lcd_write(void);
void port_write(void);
void delay_lcd(unsigned int);
unsigned long int init_command[] = {0x30,0x30,0x30,0x20,0x28,0x0c,0x06,0x01,0x80};
int main(void)
{
    SystemInit();
    SystemCoreClockUpdate();
    LPC_GPIO0->FIODIR = DT_CTRL | RS_CTRL | EN_CTRL;
    flag1 =0;
    for (i=0; i<9;i++)
    {
        temp1 = init_command[i];
        lcd_write();
    }
    flag1 =1; //DATA MODE
    i =0;
    while (msg[i] != '\0')
    {
        temp1 = msg[i];
        lcd_write();
        i+= 1;
    }
    while(1);
}

void lcd_write(void)
{
    flag2 = (flag1 == 1) ? 0 : ((temp1 == 0x30) || (temp1 == 0x20)) ? 1 : 0;
    temp2 = temp1 & 0xf0;

```

```

    temp2 = temp2 << 19;
    port_write();
if (flag2==0)
{
    temp2 = temp1 & 0x0f; //26-4+1
    temp2 = temp2 << 23;
    port_write();
}
}

void port_write(void)

{

    LPC_GPIO0->FIOPIN = temp2;
if (flag1 == 0)
    LPC_GPIO0->FIOCLR = RS_CTRL;
else
    LPC_GPIO0->FIOSET = RS_CTRL;

    LPC_GPIO0->FIOSET = EN_CTRL;
    delay_lcd(25);
    LPC_GPIO0->FIOCLR = EN_CTRL;
    delay_lcd(30000);

}

void delay_lcd(unsigned int r1)
{
    unsigned int r;
    for(r=0;r<r1;r++);

    return;
}

```

### Components required

- |                                                         |       |
|---------------------------------------------------------|-------|
| • ALS-SDA-ARMCTXM3-01 :                                 | 1 No. |
| • Power supply (+5V) :                                  | 1 No. |
| • Cross cable for programming and serial communication: | 1 No  |



- One working USB port in the host computer system and PC for downloading the software.
- 10 core FRC cables of 8 inch length 2 No
- USB to B type cable 1 No

**Hardware setup:**

Connect 10 pin FRC cable from CND to CNAD. Short the jumper JP16 & JP5. Use POT3 for contrast adjustment.

**Working procedure:** After software download and hardware setup, press the reset. A fixed message will display on LCD.

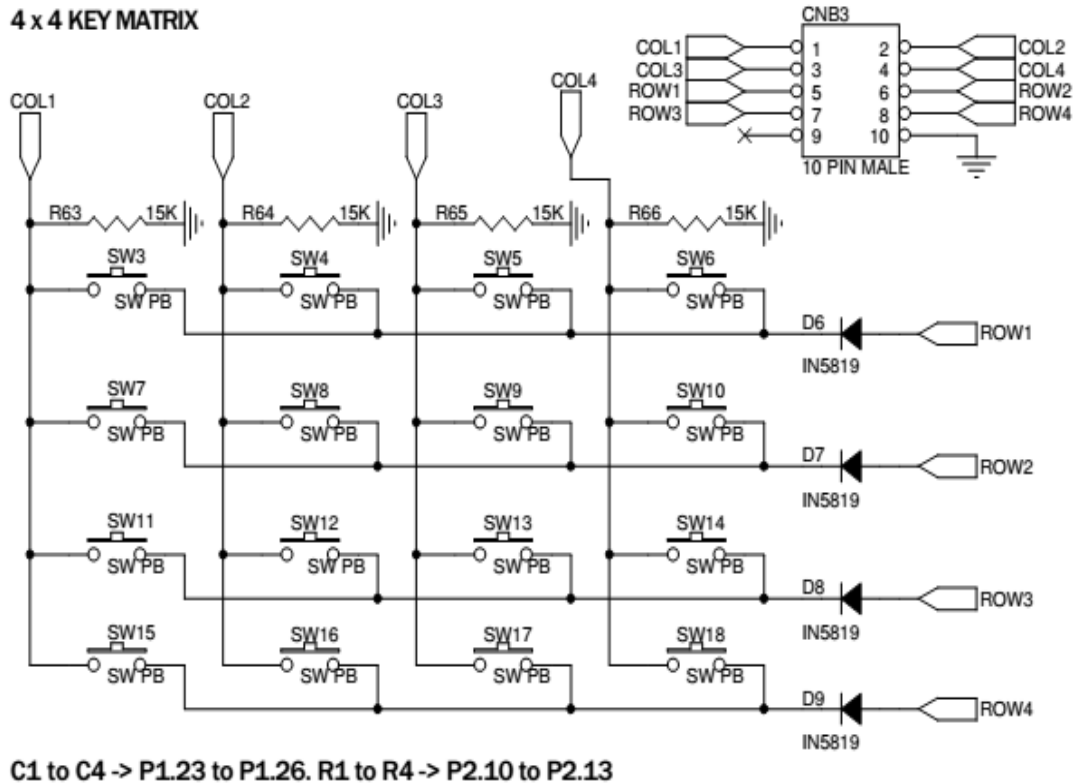
**Exercise Questions:**

1. Simulate DIE tossing on LCD

Hint: Program reads the external interrupt using the key SW2. A random number between 0-6 should be displayed on the LCD upon keypress.

**Keyboard connection:** The switches SW3 to SW18 are organized as 4 rows X 4 columns matrix. One end of all the switches are configured as columns. The other end of the matrix configured as rows. A row line will be always an output from the controller. Column lines are pulled to ground. A high level sent from the row will appear at column end if the switch is pressed.

Connector CNB3 is used for interfacing this block with controller. At the controller end any connector can be used to interact this connector CNB3.

**4 x 4 KEY MATRIX**

**C1 to C4 -> P1.23 to P1.26. R1 to R4 -> P2.10 to P2.13**

**Sample program:** To read a key from the matrix keyboard and display its key code on the LCD.

```
#include <LPC17xx.h>
void scan(void);

unsigned char col,row,var,flag,key,*ptr;
unsigned long int i,var1,temp,temp3;
int main(void)
{
    SystemInit();
    SystemCoreClockUpdate();

    LPC_PINCON->PINSEL3 &= 0xFFC03FFF; //P1.23 to P1.26 MADE
                                         //GPIO
    LPC_PINCON->PINSEL3 &= 0xF00FFFFF; //P2.10 to P2.13 made
```

```
                                //GPIO
LPC_GPIO2->FIODIR |= 0x00003C00; //made output P2.10 to
                                //P2.13 (rows)
LPC_GPIO1->FIODIR &= 0xF87FFFFFFF; //made input P1.23 to
                                //P1.26 (cols)
while(1)
{
    while(1)
    {
        for(row=1;row<5;row++)
        {
            if(row == 1)
                var1 = 0x00000400;
            else if(row == 2)
                var1 = 0x00000800;
            else if(row == 3)
                var1 = 0x00001000;
            else if(row == 4)
                var1 = 0x00002000;

            temp = var1;

            LPC_GPIO2->FIOCLR = 0x00003C00;
            LPC_GPIO2->FIOSET = var1;

            flag = 0;
            scan();
            if(flag == 1)
                break;

        } //end for(row=1;row<5;row++)

        if(flag == 1)
            break;

    } //2nd while(1)
}

void scan(void)
{
    unsigned long temp3;

    temp3 = LPC_GPIO0->FIOPIN;
    temp3 &= 0x0780000;
    if(temp3 != 0x00000000)
```

```

    {
        flag = 1;
        if (temp3 ==0x00800000)
            col=0;
        else if (temp3==0x01000000)
            col=1;
        else if (temp3==0x00200000)
            col=2;
        else if (temp3==0x04000000)
            col=3;

        }//1st if(temp3 != 0x00000000)
    }//end scan
    Display(key)    //write display function to display the keycode
                   //on the LCD or on seven segment display
}

```

### Components required

- ALS-SDA-ARMCTXM3-01 : 1 No.
- Power supply (+5V) : 1 No.
- Cross cable for programming and serial communication : 1 No
- One working USB port in the host computer system and PC for downloading the software.
- 10 core FRC cables of 8 inch length 2 No
- USB to B type cable 1 No

**Hardware setup:** Connect 10 core FRC cable from CNB to CNB3, short JP4(1, 2)  
Connect another 10 core FRC cable from CND to CNAD, Short the jumper JP16 & JP5. Use POT3 for contrast.

**Working procedure:** After software download and hardware setup, use the reset. Identity of key pressed (0 to F) will be displayed on LCD.

### Exercise questions:

1. Write a program to input an expression of the type A operator B =, from the key board, where A and B are the single digit BCD numbers and operator may be + or - .Display the result on the LCD.

## Lab 10

### Analog to Digital Convertor (ADC)

**Aim:** To understand the working of a 12 bit internal Analog-to-Digital Converter (ADC).

**Introduction:** The LPC1768 contains a single 12-bit successive approximation ADC with eight channels and DMA support. 12-bit ADC with input multiplexing among eight pins, conversion rates up to 200 kHz, and multiple result registers. The 12-bit ADC can be used with the GPDMA controller. On board there are two interfaces for internal ADC's. AD0.5 (pin P1.31) of controller is used to convert the analog input voltage varied using POT1 to digital value. AD0.4(Pin 1.30) used convert the analog voltage varied using POT4. A input voltage range of 0 to 3.3V is accepted. 000 to FFF is the converted digital voltage range here. Short JP18 (2, 3) to use AD0.4.

**Sample program:** To configure and read analog data from ADC channel no 5, and display the digital data on the LCD. Students have to write the definitions for LCD related function. The following program assumes that LCD function definitions are available in a user defined header file "AN\_LCD.h".

```
#include<LPC17xx.h>
#include<stdio.h>
#include"AN_LCD.h"
#define      Ref_Vtg          3.300
#define      Full_Scale 0xFFFF          //12 bit ADC

int main(void)
{
    unsigned long adc_temp;
    unsigned int i;
    float in_vtg;
    unsigned char vtg[7],dval[7];
    unsigned char Msg3[11] = {"ANALOG IP:"};
    unsigned char Msg4[12] = {"ADC OUTPUT:"};

    SystemInit();
    SystemCoreClockUpdate();
```

```

LPC_SC->PCONP |= (1<<15); //Power for GPIO block
lcd_init();
LPC_PINCON->PINSEL3 |= 0xC0000000; //P1.31 as AD0.5
LPC_SC->PCONP |= (1<<12); //enable the peripheral ADC

SystemCoreClockUpdate();

temp1 = 0x80;
lcd_com();
delay_lcd(800);
lcd_puts(&Msg3[0]);

temp1 = 0xC0;
lcd_com();
delay_lcd(800);
lcd_puts(&Msg4[0]);

while(1)
{
    LPC_ADC->ADCR = (1<<5)|(1<<21)|(1<<24); //0x01200001;
    //ADC0.5, start conversion and operational
    //for(i=0;i<2000;i++); //delay for conversion
    while((adc_temp = LPC_ADC->ADGDR) == 0x80000000);
    //wait till 'done' bit is 1, indicates conversion complete
    adc_temp = LPC_ADC->ADGDR;
    adc_temp >>= 4;
    adc_temp &= 0x00000FFF; //12 bit ADC
    in_vtg = (((float)adc_temp *
(float)Ref_Vtg))/((float)Full_Scale); //calculating input analog
//voltage
    sprintf(vtg,"%3.2fV",in_vtg);
    //convert the readings into string to display on LCD
    sprintf(dval,"%x",adc_temp);
    for(i=0;i<2000;i++);

    temp1 = 0x8A;
    lcd_com();
    delay_lcd(800);
    lcd_puts(&vtg[0]);

    temp1 = 0xCB;
    lcd_com();
    delay_lcd(800);
    lcd_puts(&dval[0]);
}

```

```
        for(i=0;i<200000;i++);  
        for(i=0;i<7;i++)  
        vtg[i] = dval[i] = 0x00;  
        adc_temp = 0;  
        in_vtg = 0;  
    }  
}
```

### Components required

- ALS-SDA-ARMCTXM3-01 : 1 No.
- Power supply (+5V) : 1 No.
- Cross cable for programming and serial communication : 1 No
- One working COM port (Ex: COM1) in the host computer system and PC for downloading the software.
- 10 core FRC cables of 8 inch length 2 No
- USB to B type cable 1 No

**Hardware Setup:** Do the setup related to LCD

**Working procedure:** Vary POT1 and observe the corresponding analog and digital voltage values on LCD.

### Exercise question

- a. Write a c program to display the digital value representing the difference in analog voltages at ADC channel 4 and channel 5 on LCD.

## Lab 11

### Pulse Width Modulation (PWM) Interfacing

**Aim:** To interface and understand the working of PWM.

**Introduction:** The PWM is based on the standard Timer block and inherits all of its features, although only the PWM function is pinned out on the LPC1768. The Timer is designed to count cycles of the system derived clock and optionally switch pins, generate interrupts or perform other actions when the specified timer values occur, based on seven match registers. The PWM function is in addition to these features, and is based on match register events. A PWM output from the controller can be observed as an intensity variation of the LED LD10.

**Sample program:** To vary the intensity of an LED using PWM.

```
#include <LPC17xx.H>

void pwm_init(void);
void PWM1_IRQHandler(void);

unsigned long int i;
unsigned char flag,flag1;

int main(void)
{
    SystemInit();
    SystemCoreClockUpdate();
    pwm_init();

    while(1)
    {
        for(i=0;i<=1000;i++); // delay
    } //end of while

} //end of main
```



```

void pwm_init(void)
{
    LPC_SC->PCONP |= (1<<6);           //PWM1 is powered
    LPC_PINCON->PINSEL3 &= ~(0x0000C000); //cleared if any other
                                           //functions are enabled
    LPC_PINCON->PINSEL3 |= 0x00008000; //pwm1.4 is selected for the pin
                                           //P1.23

    //LPC_PWM1->PR = 0x00000000; //Count frequency : Fpclk
    LPC_PWM1->PCR = 0x00001000; //select PWM1 single edge
    LPC_PWM1->MCR = 0x00000003; //Reset and interrupt on PWMMR0
    LPC_PWM1->MR0 = 30000; //setup match register 0 count
    LPC_PWM1->MR4 = 0x00000100; //setup match register MR1
    LPC_PWM1->LER = 0x000000FF; //enable shadow copy register
    LPC_PWM1->TCR = 0x00000002; //RESET COUNTER AND PRESCALER
    LPC_PWM1->TCR = 0x00000009; //enable PWM and counter

    NVIC_EnableIRQ(PWM1_IRQn);
    return;
}

void PWM1_IRQHandler(void)
{
    LPC_PWM1->IR = 0xff; //clear the interrupts

    if(flag == 0x00)
    {
        LPC_PWM1->MR4 += 100;
        LPC_PWM1->LER = 0x000000FF;

        if(LPC_PWM1->MR4 >= 27000)
        {
            flag1 = 0xff;

```

```

    flag = 0xff;
    LPC_PWM1->LER = 0x000000ff;
    }
}
else if(flag1 == 0xff)
{
    LPC_PWM1->MR4 -= 100;
    LPC_PWM1->LER = 0x000000ff;

    if(LPC_PWM1->MR4 <= 0x500)
    {
        flag = 0x00;
        flag1 = 0x00;
        LPC_PWM1->LER = 0x000000ff;
    }
}
}

```

**Hardware setup:** Connect 10 pin FRC cable from CNB to CNB1.

**Working procedure:** As the pulse width varies, intensity of LED LD10 varies. Observe the pulses at TP5. Observe the amplitude level at TP6.

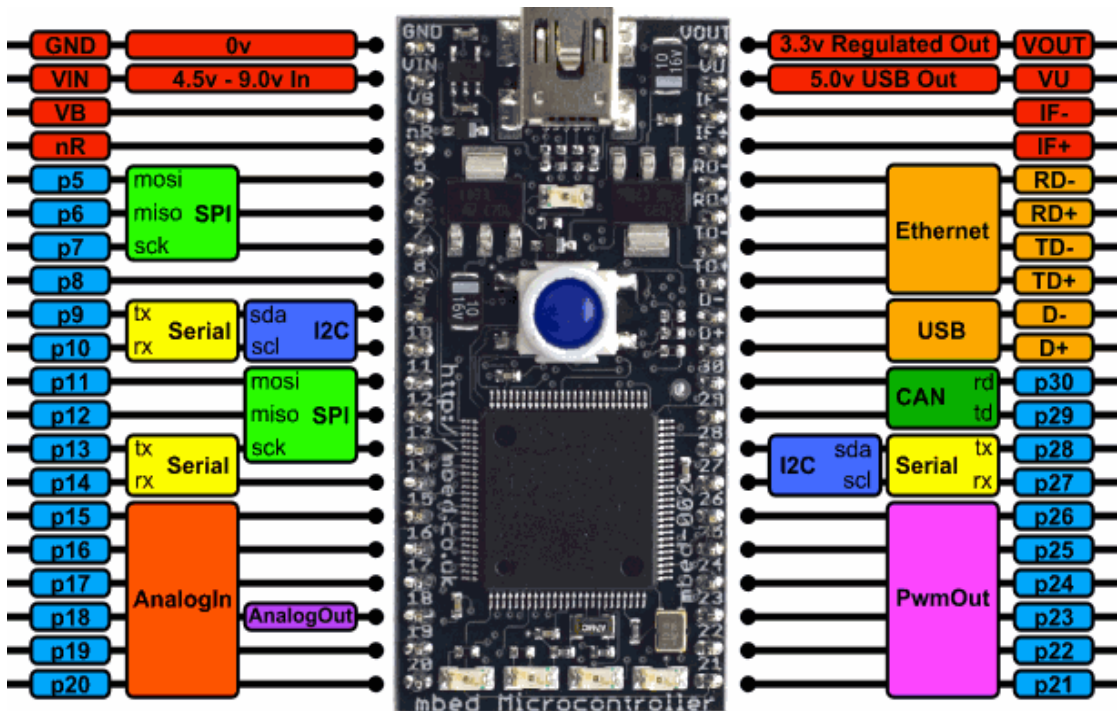
### Exercise question:

Write a program to set the following intensity levels to the LED connected to PWM output. Use ROW-0 of keyboard for intensity variation

| Intensity level | Key pressed |
|-----------------|-------------|
| 10%             | 0           |
| 25%             | 1           |
| 50%             | 2           |
| 75%             | 3           |

**Aim:** Familiarize with ARM mbed NXP LPC1768 prototyping board by interfacing with LEDs.

**Introduction:** The mbed Microcontrollers are a series of ARM microcontroller development boards designed for rapid prototyping. The mbed NXP LPC1768 Microcontroller is designed for prototyping all sorts of devices, especially those including Ethernet, USB, and the flexibility of lots of peripheral interfaces and FLASH memory. It is packaged as a small DIP form-factor for prototyping with through-hole PCBs, stripboard, and breadboard. It includes a built-in USB FLASH programmer. The pinout diagram in Figure 12.1 shows the commonly used interfaces and their locations. Note that all the numbered pins (p5-p30) can also be used as DigitalIn and DigitalOut interfaces.



### Figure 12.1 Pinout Diagram of mbed LPC1768 Board

**Components Required:**

- An mbed microcontroller
- A USB cable for connecting the microcontroller and PC.
- A PC with one working USB port for flashing the code.

To build the programs either using the ARM mbed Online Compiler or ARM mbed Studio could be used.

**Steps to be followed for using the online compiler:**

1. Connect the mbed microcontroller to a PC. The status light will glow, indicating power on. The PC will recognize the mbed microcontroller as a standard USB drive.
2. Go to the new USB drive, and click on MBED.HTML to open it in a web browser.
3. Create an mbed account using your learner id (Username: your registration number). To use the online compiler, you will need to verify your email address.  
URL: <https://os.mbed.com/account/signup/>
4. Click on the “Compiler” tab in the dashboard page, as shown in Figure 12.2.

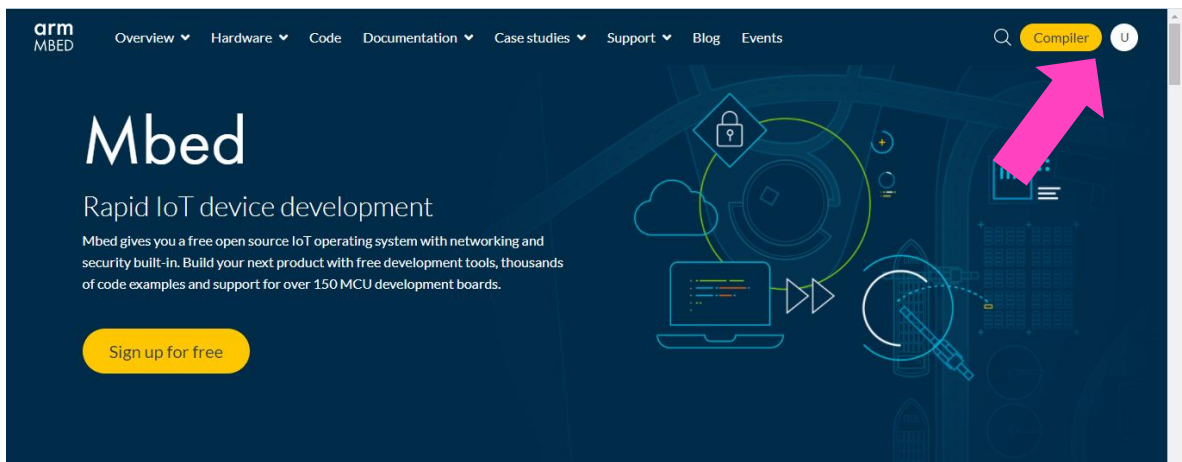


Figure 12.2 Mbed Dashboard

5. Create a new program by selecting “New Program” in the “New” tab of the online IDE. As shown in Figure 12.3, type in the program name and click “OK”.

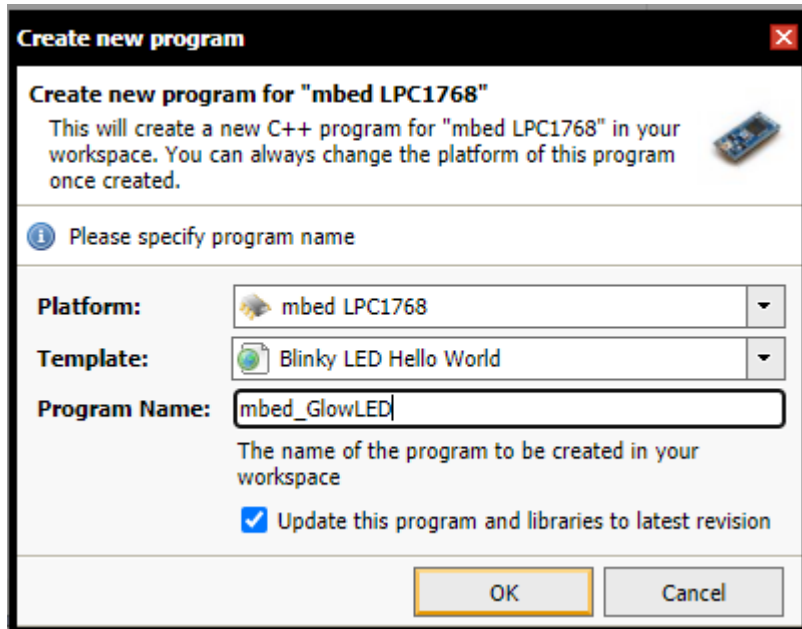


Figure 12.3 Creation of a New Program

6. In the Program Workspace, a main.cpp file will be created. Clicking on that file will reveal a sample program to blink a LED. Insert your own code in place of that. Alternatively, you could import your code into the workspace. Save your program.
7. Compile the code by clicking on the “Compile” tab, or press CTRL-D. The Online Compiler compiles the code into an executable file, which will be downloaded. When prompted, **save the file(.bin) in mbed board's USB device folder**.
8. Press reset switch on the board to complete the flashing/programming process.

**Sample Question:** Write a CPP program to blink all four LEDs in the mbed LPC1768 board.

```
/**
 * *****
 * // Program written using C++ for mbed LPC1768 board.
 * // Program to blink all four onboard LEDs.
 * // Author: *****
 * // Date: DD-Mon-YYYY
 * *****
 */
#include "mbed.h"
DigitalOut myled1(LED1);
DigitalOut myled2(LED2);
DigitalOut myled3(LED3);
DigitalOut myled4(LED4);
int main()
{
    while(1) {
        //turn on LED1 through LED4
        myled1 = 1;
        myled2 = 1;
        myled3 = 1;
        myled4 = 1;
        wait(0.5); // delay of 0.5 seconds
        //turn off LED1 through LED4
        myled1 = 0;
        myled2 = 0;
        myled3 = 0;
        myled4 = 0;
        wait(0.5); //delay of 0.5 seconds
    }
}
```

**Exercise Programs**

1. Write a CPP program to display a 4-bit binary up counter on the LEDs.
2. Write a CPP program to display a 4-bit binary down counter on the LEDs.
3. Write a CPP program to display a ring counter on the LEDs.

## Appendix A: Instructions

### Instruction Set Summary

| Mnemonic                     | Operation                                                          | Description                                                     |
|------------------------------|--------------------------------------------------------------------|-----------------------------------------------------------------|
| <a href="#"><u>ADC</u></a>   | $Rd := Rn + Op2 + C$                                               | Add with carry                                                  |
| <a href="#"><u>ADD</u></a>   | $Rd := Rn + Op2$                                                   | Add                                                             |
| <a href="#"><u>ADDS</u></a>  | $Rd := Rn + Op2$                                                   | Add and update falgs                                            |
| <a href="#"><u>ADR</u></a>   | $Rd := Rn, \text{label}$                                           | Load register with address                                      |
| <a href="#"><u>AND</u></a>   | $Rd := Rn \text{ AND } Op2$                                        | AND                                                             |
| <a href="#"><u>ANDS</u></a>  | $Rd := Rn \text{ AND } Op2$                                        | AND update flags                                                |
| <a href="#"><u>ASR</u></a>   | $Rd := Rn, \#LSB, \#width$                                         | Arithmetic shift right                                          |
| <a href="#"><u>B</u></a>     | $R15 := \text{address}$                                            | Branch                                                          |
| <a href="#"><u>BIC</u></a>   | $Rd := Rn \text{ AND NOT } Op2$                                    | Bit Clear                                                       |
| <a href="#"><u>BL</u></a>    | $R14 := \text{address of next instruction}, R15 := \text{address}$ | Branch with Link                                                |
| <a href="#"><u>BX</u></a>    | $R15 := Rn, \text{change to Thumb if address bit 0 is 1}$          | Branch and Exchange                                             |
| <a href="#"><u>CLZ</u></a>   | $Rd := \text{number of leading zeroes in } Rm$                     | Count Leading Zeroes                                            |
| <a href="#"><u>CMN</u></a>   | $CPSR \text{ flags} := Rn + Op2$                                   | Compare Negative                                                |
| <a href="#"><u>CMP</u></a>   | $CPSR \text{ flags} := Rn - Op2$                                   | Compare                                                         |
| <a href="#"><u>EOR</u></a>   | $Rd := Rn \text{ EOR } Op2$                                        | Exclusive OR                                                    |
| <a href="#"><u>LDM</u></a>   | Stack manipulation (Pop)                                           | Load multiple Registers (refer last paragraph of this appendix) |
| <a href="#"><u>LDMIA</u></a> | $LDMIA Rn!, \{\text{reglist}\}$                                    | Load multiple registers from memory                             |
| <a href="#"><u>LDR</u></a>   | $Rd := [\text{address}][31:0]$                                     | Load 32-bit word from memory.                                   |
| <a href="#"><u>LDRB</u></a>  | $Rd := \text{ZeroExtend}([\text{address}][7:0])$                   | Load register byte value to Memory.                             |
| <a href="#"><u>LDRH</u></a>  | $Rd := \text{ZeroExtend}([\text{address}][15:0])$                  | Load register 16-bit halfword value to Memory.                  |
| <a href="#"><u>MCR</u></a>   | $cRn := rRn \{<op>cRm\}$                                           | Move CPU register to coprocessor register                       |
| <a href="#"><u>MLA</u></a>   | $Rd := (Rm * Rs) + Rn$                                             | Multiply Accumulate                                             |
| <a href="#"><u>MOV</u></a>   | $Rd := Op2$                                                        | Move register or constant                                       |
| <a href="#"><u>MRS</u></a>   | $Rn := PSR$                                                        | Move PSR status flags to register                               |



|                     |                             |                                                                                                           |
|---------------------|-----------------------------|-----------------------------------------------------------------------------------------------------------|
| <b><u>MSR</u></b>   | PSR := Rm                   | Move register to PSR status flags                                                                         |
| <b><u>MUL</u></b>   | Rd := Rm * Rs               | Multiply                                                                                                  |
| <b><u>MVN</u></b>   | Rd := NOT Rm                | Move inverted register or constant                                                                        |
| <b><u>NOP</u></b>   | None                        | No operation                                                                                              |
| <b><u>ORR</u></b>   | Rd := Rn OR Op2             | OR                                                                                                        |
| <b><u>PUSH</u></b>  | PUSH {reg list}             | Push registers on to the stack pointed by R13                                                             |
| <b><u>POP</u></b>   | POP{reg. list}              | Pop registers from the stack pointed by R13                                                               |
| <b><u>RSB</u></b>   | Rd := Op2 - Rn              | Reverse Subtract                                                                                          |
| <b><u>RSC</u></b>   | Rd := Op2 - Rn - 1+Carry    | Reverse Subtract with Carry                                                                               |
| <b><u>RBIT</u></b>  | RBIT Rd, Rn                 | Reverse the bit order in a 32-bit word                                                                    |
| <b><u>REV</u></b>   | REV Rd, Rn                  | converts 32-bit big-endian data into little-endian data or 32-bit little-endian data into big-endian data |
| <b><u>ROR</u></b>   | Rd := Rd, Rs                | Rotate Rd register by Rs bits                                                                             |
| <b><u>RRX</u></b>   | Rd := Rd, Rm                | Rotate Right with Extend                                                                                  |
| <b><u>SBC</u></b>   | Rd := Rn - Op2 - 1+Carry    | Subtract with Carry                                                                                       |
| <b><u>STM</u></b>   | stack manipulation (Push)   | Store Multiple (refer last paragraph of this appendix)                                                    |
| <b><u>STR</u></b>   | <address>:=Rd               | Store register to memory                                                                                  |
| <b><u>STRB</u></b>  | [address][7:0] := Rd[7:0]   | Store register byte value to Memory.                                                                      |
| <b><u>STRH</u></b>  | [address][15:0] := Rd[15:0] | Store register 16-bit halfword value to Memory                                                            |
| <b><u>SUB</u></b>   | Rd := Rn - Op2              | Subtract                                                                                                  |
| <b><u>TEQ</u></b>   | CPSR flags:= Rn EOR Op2     | Test bitwise equality                                                                                     |
| <b><u>TST</u></b>   | CPSR flags:= Rn AND Op2     | Test bits                                                                                                 |
| <b><u>UMULL</u></b> | UMULL r0, r4, r5, r6        | Unsigned Long Multiply                                                                                    |

A conditional instruction is only executed on match of the condition flags in the Program Status Register. For example, the BEQ (B instruction with EQ condition) branches only if the Z flag is set. If the {cond} field is empty the instruction is always executed.

| {cond} Suffix | Tested Status Flags         | Description               |
|---------------|-----------------------------|---------------------------|
| <b>EQ</b>     | Z set                       | equal                     |
| <b>NE</b>     | Z clear                     | not equal                 |
| <b>CS/HS</b>  | C set                       | unsigned higher or same   |
| <b>CC/LO</b>  | C clear                     | unsigned lower            |
| <b>MI</b>     | N set                       | negative                  |
| <b>PL</b>     | N clear                     | positive or zero          |
| <b>VS</b>     | V set                       | overflow                  |
| <b>VC</b>     | V clear                     | no overflow               |
| <b>HI</b>     | C set and Z clear           | unsigned higher           |
| <b>LS</b>     | C clear or Z set            | unsigned lower or same    |
| <b>GE</b>     | N equals V                  | signed greater or equal   |
| <b>LT</b>     | N not equal to V            | signed less than          |
| <b>GT</b>     | Z clear AND (N equals V)    | signed greater than       |
| <b>LE</b>     | Z set OR (N not equal to V) | signed less than or equal |
| <b>AL</b>     | (ignored)                   | always (usually omitted)  |

## Addressing Mode for LDM and STM

The instructions LDM and STM provide four different addressing modes. The addressing mode specifies the behavior of the base register and is explained in the following table.

| Addressing Mode | Description                                           |
|-----------------|-------------------------------------------------------|
| IA              | Increment base register after instruction execution.  |
| IB              | Increment base register before instruction execution. |
| DA              | Decrement base register after instruction execution.  |
| DB              | Decrement base register before instruction execution. |

### Examples:

```
STMDB    R2!, {R4, R5, LR}
LDMIA    R0!, {R1-R5}
STMDB    R6!, {R0, R1, R5}
```

**Appendix B: Addressing modes**

| Name                                   | Alternative Name                          | ARM Examples                   |
|----------------------------------------|-------------------------------------------|--------------------------------|
| Register to register                   | Register direct                           | MOV R0, R1                     |
| Absolute                               | Direct                                    | LDR R0, MEM                    |
| Literal                                | Immediate                                 | MOV R0, #15<br>ADD R1, R2, #12 |
| Indexed, base                          | Register indirect                         | LDR R0, [R1]                   |
| Pre-indexed,<br>base with displacement | Register indirect<br>with offset          | LDR R0, [R1, #4]               |
| Pre-indexed,<br>autoindexing           | Register indirect<br>pre-incrementing     | LDR R0, [R1, #4]!              |
| Post-indexing,<br>autoindexed          | Register indirect<br>post-increment       | LDR R0, [R1], #4               |
| Double Reg indirect                    | Register indirect<br>Register indexed     | LDR R0, [R1, R2]               |
| Double Reg indirect<br>with scaling    | Register indirect<br>indexed with scaling | LDR R0, [R1, r2, LSL #2]       |
| Program counter relative               |                                           | LDR R0, [PC, #offset]          |

**Literal Addressing**

In this addressing mode data is a part of instruction. ‘#’ symbol is used to indicate the data. ARM and Thumb instructions can only be 32 bits wide. You can use a **MOV** or **MVN** instruction to load a register with an immediate value from a range that depends on the instruction set. Certain 32-bit values cannot be represented as an immediate operand to a single 32-bit instruction, although you can load these values from memory in a single

instruction. you can load any 32-bit immediate value into a register with two instructions, a **MOV** followed by a **MOVT**. Or, you can use a pseudo-instruction, **MOV32**, to construct the instruction sequence for you. You can also use the **LDR** pseudo-instruction to load immediate values into a register

| Examples                             | Meaning                                                   |
|--------------------------------------|-----------------------------------------------------------|
| <code>CMP R0, #22</code>             | ;Compare Register content R0 with 22                      |
| <code>ADD R1, R2, #18</code>         | ;Add the content of R2 and 18 then store the result in R1 |
| <code>MOV R1, #30</code>             | ;copy the data 30 into register R1                        |
| <code>MOV R1, #0Xff</code>           | ;copy the data ff in hexadecimal into R1                  |
| <code>MOV R2, #0xFF0000FF</code>     |                                                           |
| <code>AND R0, R1, #0xFF000000</code> |                                                           |
| <code>CMN R0, #6400</code>           | ; update the N, Z, C and V flags                          |
| <code>CMPGT SP, R7, LSL #2</code>    | ; update the N, Z, C and V flags                          |

- **MOV** can load any 8-bit immediate value, giving a range of **0x0-0xFF** (0-255). It can also rotate these values by any even number. These values are also available as immediate operands in many data processing operations, without being loaded in a separate instruction.
- **MVN** can load the bitwise complements of these values. The numerical values are  $-(n+1)$ , where **n** is the value available in **MOV**.
- A **MOVT** instruction that can load any value in the range **0x0000** to **0xFFFF** into the most significant half of a register, without altering the contents of the least significant half.
- The **LDR Rd,=const** pseudo-instruction generates the most efficient single instruction to load any 32-bit number

**Introduction to Register Indirect Addressing :** Register indirect addressing means that the location of an operand is held in a register. It is also called indexed addressing or base addressing.

Register indirect addressing mode requires three read operations to access an operand. It is very important because the content of the register containing the pointer to the operand can be modified at runtime. Therefore, the address is a variable that allows the access to the data structure like arrays.

- Read the instruction to find the pointer register
- Read the pointer register to find the operand address
- Read memory at the operand address to find the operand

Some examples of using register indirect addressing mode:

```
LDR    R2, [R0]        ; Load R2 with the word pointed by R0
```

```
-----
STR    R2, [R3]        ; Store the word in R2 in the location
                        ; pointed by R3
-----
```

`LDR Rd, =label` can load any 32-bit numeric value into a register. It also accepts PC-relative expressions such as labels, and labels with offsets

### Register Indirect Addressing with an Offset

ARM supports a memory-addressing mode where the effective address of an operand is computed by adding the content of a register and a literal offset coded into load/store instruction. For example,

| Instruction       | Effective Address                                         |
|-------------------|-----------------------------------------------------------|
| LDR R0, [R1, #20] | R1 + 20 ; loads R0 with the word<br>; pointed at by R1+20 |

### ARM's Autoindexing Pre-indexed Addressing Mode

This is used to facilitate the reading of sequential data in structures such as arrays, tables, and vectors. A pointer register is used to hold the base address. An offset can be added to achieve the effective address. For example,

| Instruction       | Effective Address |                                                                                                      |
|-------------------|-------------------|------------------------------------------------------------------------------------------------------|
| LDR R0, [R1, #4]! | $R1 + 4$          | ; loads R0 with the word<br>; pointed at by R1+4 then<br>; update the pointer<br>; by adding 4 to R1 |

### ARM's Autoindexing Post-indexing Addressing Mode

This is similar to the above, but it first accesses the operand at the location pointed by the base register, then increments the base register. For example,

| Instruction      | Effective Address |                                                                                                    |
|------------------|-------------------|----------------------------------------------------------------------------------------------------|
| LDR R0, [R1], #4 | R1                | ; loads R0 with the word<br>; pointed at by R1 then<br>; update the pointed by<br>; adding 4 to R1 |

### Program Counter Relative (PC Relative) Addressing Mode

Register R15 is the program counter. If you use R15 as a pointer register to access operand, the resulting addressing mode is called PC relative addressing. The operand is specified with respect to the current code location. Please look at this example,

| Instruction        | Effective Address |                                                   |
|--------------------|-------------------|---------------------------------------------------|
| LDR R0, [R15, #24] | $R15 + 24$        | ; loads R0 with the word<br>; pointed at by R1+24 |

## APPENDIX C

### GPIO extension connectors:

There are four 10 pin FRC type male connectors, they extend the controllers general purpose port lines for the use of user requirements. Details on each connector are given below:

**CNA** – 10 pin male box type FRC connector. Port lines P0.4 to P0.11 from controller are terminated in this connector. They can be extended to interface few on board or external peripherals. The pins mentioned in the above table are configured to work as a GPIO's at power on. Other alternate functions on those pins need to be selected using respective PINSEL registers.

| Pin CNA | PIN LPC1768 | Description                 |
|---------|-------------|-----------------------------|
| 1       | 81          | P0.4/I2SRX_CLK/RD2/CAP2.0   |
| 2       | 80          | P0.5/I2SRX_WS/TD2/CAP2.1    |
| 3       | 79          | P0.6/I2SRX_SDA/SSEL1/MAT2.0 |
| 4       | 78          | P0.7/I2STX_CLK/SCK1//MAT2.1 |
| 5       | 77          | P0.8/I2STX_WS/MISO1/MAT2.2  |
| 6       | 76          | P0.9/I2STX_SDA/MOSI1/MAT2.3 |
| 7       | 48          | P0.10/TXD2/SDA2/MAT3.0      |
| 8       | 49          | P0.11/RXD2/SCL2/MAT3.1      |
| 9       | -           | No connection               |
| 10      | -           | Ground                      |

**CNB** – 10 pin male box type FRC connector. Port lines from P1.23 to P1.26 and P2.10 to

P2.13 are terminated in this connector.

Description of the connector **CNB**:

| <b>Pin CNB</b> | <b>Pin LPC1768</b> | <b>Description</b>        |
|----------------|--------------------|---------------------------|
| 1              | 37                 | P1.23/MCI1/PWM1.4/MISO0   |
| 2              | 38                 | P1.24/MCI2/PWM1.5/MOSI0   |
| 3              | 39                 | P1.25/MCOA1/MAT1.1        |
| 4              | 40                 | P1.26/MCOB1/PWM1.6/CAP0.0 |
| 5              | 53                 | P2.10/EINT0/NMI           |
| 6              | 52                 | P2.11/EINT1/I2STX_CLK     |
| 7              | 51                 | P2.12/EINT2/I2STX_WS      |
| 8              | 50                 | P2.13/EINT3/I2STX_SDA     |
| 9              | -                  | No connection             |
| 10             | -                  | Ground                    |

**CNC** – 10 pin male box type FRC connector. Port lines from P0.15 to P0.22 and P2.13 are terminated in this connector.

| <b>Pin CNC</b> | <b>Pin LPC1768</b> | <b>Description</b>    |
|----------------|--------------------|-----------------------|
| 1              | 62                 | P0.15/TXD1/SCK0/SCK   |
| 2              | 63                 | P0.16/RXD1/SSEL0/SSEL |
| 3              | 61                 | P0.17/CTS1/MISO0/MISO |
| 4              | 60                 | P0.18/DCD1/MOSI0/MOSI |
| 5              | 59                 | P0.19/DSR1/SDA1       |
| 6              | 58                 | P0.20/DTR1/SCL1       |
| 7              | 57                 | P0.21/RI1/RD1         |
| 8              | 56                 | P0.22/RTS1/TD1        |
| 9              | 50                 | P2.13/I2STX_SDA       |
| 10             | -                  | Ground                |



**CND** – 10 pin male box type FRC connector. Port lines from P0.23 to P0.28 and P2.0 to P2.1 are terminated in this connector.

| <b>Pin CND</b> | <b>Pin LPC1768</b> | <b>Description</b>           |
|----------------|--------------------|------------------------------|
| 1              | 9                  | P0.23/AD0.0/I2SRX_CLK/CAP3.0 |
| 2              | 8                  | P0.24/AD0.1/I2SRX_WS/CAP3.1  |
| 3              | 7                  | P0.25/AD0.2/I2SRX_SDA/TXD3   |
| 4              | 6                  | P0.26/AD0.3/AOUT/RXD3        |
| 5              | 25                 | P0.27/SDA0/USB/SDA           |
| 6              | 24                 | P0.28/SCL0/USB_SCL           |
| 7              | 75                 | P2.0/PWM1.1/TXD1             |
| 8              | 74                 | P2.1/PWM1.2/RXD1             |
| 9              | -                  | No connection                |
| 10             | -                  | Ground                       |